

Voice-first indoor guidance assistant

Audio analysis, delivery plan and Kanban stories

Itnig hackathon | 9 October 2026 | Team working brief

The idea

A browser-based assistant uses the phone camera, microphone and spoken feedback to help a blind or visually impaired person explore an unfamiliar indoor space and reach a requested destination.

The first demo

Demonstrate one controlled route at Itnig, with a small set of predefined destinations. The discussion proposes the counter to order a coffee, then the bathroom; a meeting room is another example. A predefined script comes first. Open-ended destinations and free exploration come later.

How to read this brief

Recorded: an agreement or proposal supported by the audio, with elapsed timestamps. Recommended: a delivery choice added to make the work executable. These recommendations are not claims about what the team agreed.

Source: Itnig Spaces copy.m4a, duration 14:48. Automatic transcription used for analysis. Some speech overlaps and technical terms are unclear. Speaker names and individual assignments are not reliably attributable; verify them before assigning work.

Success criterion - recommended

On a real phone, start the session, state one supported destination, receive a short spoken instruction informed by a current camera image, and stop the session. Demonstrate this on a supervised route. This prototype does not establish safe independent navigation.

What the team decided

Decision	Evidence / qualification
Voice-first web app	00:18-00:44; 07:29-07:52. Camera input, microphone requests and speaker output; minimal visual interface.
Constrain the first route	01:36-02:35; 03:00-03:18. Counter / bathroom at the venue, predefined script first, limited destinations; tags at key places are suggested.
Start / stop with a double tap	07:03-07:49; 08:24-08:35. Explicitly prioritized before real-time voice interaction. Gesture is an MVP proposal; accessibility behavior still needs testing.
Three functional pipelines	02:35-04:05; 08:59-09:14. Voice input, navigation engine / LLM-VLA coordination, and TTS output. Work can proceed simultaneously.
Include a developer debug mode	08:39-08:55. Show useful information and logs in the app to speed diagnosis and agent-assisted development.
Minimal interface	05:35-07:10. Simple activation is favored. Segmentation display is discussed for developers; helper setup is raised but not clearly accepted as MVP scope.

Core user flow - recorded

Open the web app → double tap → hear a prompt asking where to go → speak the destination → send the captured request to a server → process it with the navigation / AI pipeline → speak the guidance. Double tap again to stop. The precise recording-end behavior is not settled.

Exploration ideas - not selected technology

Vision-language-action models (VLAs), an Omni-VLA-like approach and 2D navigation are discussed because images alone do not provide a map. Voice API services versus hosting Voxtral are alternatives. Showing segmentation, venue tags and a helper-facing setup view are ideas, not established requirements.

Infrastructure and open questions

Recorded infrastructure direction

Start locally, then optimize. At 10:48-11:24 the team discusses serving small models from a teammate's computer through a tunnel, using that for the first iteration, and considering other inference options tomorrow if something works tonight. APIs, remote inference and GPU hosting remain options rather than a finalized deployment architecture.

Budget: a EUR 50 pool is proposed and receives agreement around 10:11-10:28. The following split is poorly transcribed; EUR 12.50 each for four people is an arithmetic interpretation, not a reliably captured quotation. Confirm the cap and split before spending. Hardware / inference price estimates in the conversation are unverified.

Decide in the first 30 minutes - recommended

1. Pick one route, start point and destination.
2. Name one owner per pipeline and one integration lead.
3. Choose a working speech input/output provider and one navigation candidate.
4. Agree the budget cap and who can provision inference.
5. Fix the shared request/response contract before developing separately.

Unresolved question	Recommended default for the demo
Does the engine understand route progress?	Use explicit route steps and observable landmarks. Do not claim general mapless navigation.
VLA model, dataset and hosting?	Time-box a model spike; choose by an actual venue test, latency and deployment effort.
When is a voice request complete?	Use a bounded recording with an explicit stop control; defer always-listening interaction.
Is double tap compatible with assistive tools?	Test with a screen reader; provide one large, labeled Start / Stop control as well.
How are errors communicated?	Speak that guidance is unavailable, halt the session, and allow retry / stop.
Who owns each workstream?	Role assignments below are proposals; the audio does not reliably name assignees.

Build plan and responsibilities

Recommended sequence from kickoff. Estimates are planning ranges, not recorded commitments. Workstreams can run in parallel after the shared contract is agreed.

Phase	Outcome / gate	Work split
0 0-30 min	Freeze one route, contracts and owners. Confirm real-phone camera/mic access.	All; integration lead drives.
1 30-90 min	Speech input, image-to-guidance and spoken output each work independently. Navigation candidate passes a venue image test or fallback is chosen.	Input owner; two people on engine evaluation (suggested in audio); output/UI owner.
2 90 min-3 h	One full request works on a phone through the local inference endpoint. Start / stop and debug trace work.	Integration lead plus each pipeline owner.
3 3-5 h	Supervised venue route, bounded retries, uncertain-scene behavior and measured latency.	Engine owner plus a tester; input/output owners fix failures.
4 Next build block	Improve inference hosting only if needed; stabilize and rehearse. Stretch work starts only after MVP passes.	Infrastructure owner; demo lead; all review.

Suggested four-person split

A - input and integration: capture microphone/camera data, define API contract, connect the browser to inference.

B - navigation: evaluate candidate models, implement route decisions and uncertainty handling.

C - navigation and infrastructure: partner on model evaluation, run local serving/tunnel, measure latency and cost.

D - output and interface: TTS, accessible activation/stop, debug panel and demo presentation. Choose actual names together.

Minimum shared contract - recommended

Request: session ID, supported destination, current route step, audio or transcript, image, capture timestamp.

Response: request ID, status, short guidance text, action (wait / turn / continue / arrived / stop), next route step, and debug timing. Only advance when the chosen route logic can justify it; reject stale responses after stop.

Kanban board: proposed starting state

This is a recommended starting board, not a report of completed work. Nothing is marked Done because the recording contains no verified implementation results. Move a card only when its exit criteria are met.

Backlog	Ready	In progress	Review / Done
S05 Route engine S06 TTS S07 Haptics S08 Debug trace S09 Full integration S10 Failure handling S11 Venue test S12 Demo release	S01 Scope + contract S02 Phone capture S03 Speech input S04 Model spike	Empty Pull after agreeing owners. Limit: one card per owner; model spike may have two.	Empty Review: evidence + peer check. Done: acceptance criteria passed on target phone.

Pull order and dependencies

First: S01, then S02 / S03 / S04 in parallel. Next: S05 and S06, with S08 alongside. Integrate: S09, then S10 and S11. Ship: S12. S07 is a bounded output spike and must not block the voice demo.

Priority and effort

P0: needed for the controlled voice demo. P1: useful agreed exploration or supporting feature; cut if it threatens integration. Effort ranges are per-card working estimates, can overlap, and exclude unexpected model/hosting problems.

Scope cut line

Keep one route, one navigation implementation and one speech path. Defer free exploration, arbitrary destinations, always-listening commands, broad environment mapping, elaborate helper setup and visual segmentation polish. Investigate simple haptics only after spoken output works.

Additional recorded ideas

11:34-12:58: vibration as directional or stop feedback is debated, including one/two pulses and continuous vibration. The team agrees it needs definition; no stable vocabulary is selected.
13:10-14:12: run-log storage and shared access are discussed. Local storage and Google Cloud credits are alternatives; no cloud service is definitively selected. Credit amount and eligibility are participant claims, not verified infrastructure facts.

Definition of Done - recommended

The story passes its acceptance criteria on the actual demo phone; another teammate checks the result; errors are observable; Stop halts pending output; and evidence (a short log or clip) is attached to the card. Avoid storing raw camera/audio data in routine logs.

Story cards 1-4

S01 | Scope and shared contract

P0 | 20-30 min | Integration lead | No dependency

As a teammate, I need one route and a shared contract so the three pipelines can connect without guessing.

Acceptance: Record start point, destination, allowed route steps and fallback. Name pipeline owners. Commit request/response fixtures with session/request IDs and error states. Confirm budget cap and who provisions inference.

S02 | Camera, microphone and activation

P0 | 45-75 min | Input owner | S01

As a user, I can activate the app and capture the inputs needed for guidance.

Acceptance: On the target phone, camera and mic permission success/denial are handled. A fresh frame and bounded audio recording can be captured. Start/Stop is labeled and usable; test the proposed double tap with a screen reader. Stop releases capture.

S03 | Speech to supported destination

P0 | 45-90 min | Input owner | S01; live capture uses S02

As a user, I can say where I want to go and hear when the request was understood.

Acceptance: Run a real venue recording through the chosen API or local speech model. Map supported destination phrasing to a fixed destination ID. Empty, noisy or unsupported requests prompt retry rather than starting guidance. Document one measured turnaround.

S04 | Navigation model feasibility spike

P0 | 60-90 min cap | Engine pair | S01

As the team, we need evidence that an engine can give usable guidance for our chosen route.

Acceptance: Test candidate input images and destination prompts from the venue. Compare response usefulness, latency and setup effort. Select one engine and record its limitations. If no candidate passes, choose a clearly labeled scripted route with AI scene assistance; do not claim autonomous navigation.

Story cards 5-8

S05 | Route-aware navigation engine

P0 | 60-120 min | Engine owner | S04 + S01

As a user, I receive one short instruction appropriate to my current route step.

Acceptance: Combine destination, fresh image and route state. Return structured action, concise guidance and next state. Handle uncertain scenes with wait/stop and a request for a clearer view. Demonstrate an arrived state. Reject expired input instead of inventing route progress.

S06 | Spoken prompt and guidance

P0 | 30-60 min | Output/UI owner | S01

As a user, I hear the destination prompt and guidance without reading the screen.

Acceptance: After user activation, speak the opening prompt and a mocked engine response on the target phone. Serialize speech so instructions do not overlap. Stop cancels queued and current output. TTS failure produces a recoverable app state.

S07 | Define and test haptic output

P1 | 30-45 min cap | Output/UI owner | S06

As a user, I may receive simple vibration cues alongside spoken instructions.

Acceptance: Check vibration support on the demo device. Define and document at most two clearly distinct cues; test comprehension with a sighted/supervised tester. Unsupported devices retain full spoken guidance. Keep directional vibration experimental until it is understood consistently.

S08 | Debug panel and shared run trace

P0 | 30-60 min | Infrastructure owner | S01

As a developer, I can see where a request failed and compare successive runs.

Acceptance: Show request ID, route step, transcript/destination, selected engine, action, stage timings and error. All teammates can access a sanitized run trace through the agreed local setup. Debug view is separate from the primary interaction; exclude secrets and raw media by default.

Story cards 9-12

S09 | Connect the full phone loop

P0 | 60-90 min | Integration lead + owners | S02/S03/S05/S06

As a user, I can ask for a supported destination and hear guidance based on the current scene.

Acceptance: Phone reaches the selected inference endpoint through an HTTPS setup. Run activation → request → camera/engine → spoken output end to end. Track latency across stages. Associate each result with the correct session/request. API credentials stay server-side.

S10 | Stop, stale output and failure recovery

P0 | 30-60 min | Integration + engine | S09

As a user, I can halt guidance and understand when it cannot continue.

Acceptance: Stop cancels capture, requests and output; late responses cannot restart speech. Handle timeout, offline state, denied permission and uncertain scene. Prevent contradictory overlapping instructions. A retry begins from an explicitly known session/route state.

S11 | Supervised venue acceptance test

P0 | 45-75 min | Tester + engine owner | S09/S10/S08

As the team, we can demonstrate the bounded route with observable evidence.

Acceptance: Complete three supervised runs on the target phone, record observed failures and latency, and fix blockers. Include an unsupported destination and uncertain scene. Set an explicit acceptable latency based on testing. Do not ask a blind person to rely on the unvalidated prototype for mobility.

S12 | Freeze and rehearse the shareable demo

P0 | 30-60 min | Demo lead + all | S11

As a judge or teammate, I can understand the value and limits in a short live demo.

Acceptance: Freeze a known working version. Rehearse a 2-3 minute scenario: ask, observe, guide, stop. Show the AI contribution and disclose any script/tags. Prepare a labeled recorded fallback, endpoint restart notes and a simple budget check. Explain free exploration and richer haptics as next steps.